PCA Analysis of the Iris Dataset

Oğuzhan Işılay

Introduction: What is PCA?

In many biological and scientific datasets, we measure **many variables at the same time**.

When a dataset has many variables, it becomes **difficult to visualize and understand**.

Principal Component Analysis (PCA) is a statistical method used to **reduce the dimensionality of data** while keeping most of the important information.

PCA creates new variables called **principal components (PCs)**.

- **PC1** captures the largest amount of variation in the dataset.
- **PC2** captures the second largest variation.
- Additional components capture smaller amounts of variation.

By plotting **PC1 and PC2**, we can visualize complex datasets in **two dimensions**.

Learning Goals

In this tutorial you will learn:

- What PCA is
- How to run PCA in R
- How to visualize PCA results
- How to interpret a PCA plot

1. Load Required Package

We will use the **ggplot2** package to create clear and professional plots.

If you do not have ggplot2 installed, run this once in the Console:

`install.packages("ggplot2")`

```
library(ggplot2)
```

2. Load and Explore the Iris Dataset

The **iris dataset** is one of the most famous datasets in **data science and statistics**.

It is included by default in **R**, so we do not need to download it.

The dataset contains measurements of **150 iris flowers**.

Species in the dataset

There are **3 different species**:

- setosa
- versicolor
- virginica

Numeric measurements

For each flower, there are **4 numeric measurements**:

- **Sepal.Length** → length of the sepal
- **Sepal.Width** → width of the sepal
- **Petal.Length** → length of the petal
- **Petal.Width** → width of the petal

Categorical variable

There is also one **categorical variable**:

- **Species** → indicates which species the flower belongs to

Load the iris dataset

```r
# Load the iris dataset (it already exists in R)
data(iris)

# Show the first 6 rows of the dataset
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```r
# Display the structure of the dataset
str(iris)
```

```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

What do we learn from this?

From this dataset we learn:

- There are **150 observations (flowers)**.
- There are **5 variables**.
- The first **4 variables are numeric**.
- The last variable (**Species**) is a **categorical variable**.

3. Prepare the Data for PCA

Before running PCA, we need to prepare the dataset.

Principal Component Analysis (PCA) works only with **numeric variables**.
Therefore we cannot include categorical variables such as **Species** in the PCA calculation.

In the iris dataset:

- The first **4 columns are numeric measurements**
- The last column (**Species**) is a **categorical variable**

For this reason, we will select only the **numeric columns**.

Select numeric variables

```r
# Load the iris dataset into the R environment
# The iris dataset already exists inside R
data(iris)

# head() shows the first 6 rows of the dataset
# This helps us quickly understand the structure of the data
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```r
# str() shows the structure of the dataset
str(iris)
```

```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

What do we learn from this?

From this dataset we learn:

- There are **150 observations (flowers)**.
- There are **5 variables**.
- The first **4 variables are numeric**.
- The last variable (**Species**) is a **categorical variable**.

3. Prepare the Data for PCA

Before running PCA, we need to prepare the dataset.

Principal Component Analysis (PCA) works only with **numeric variables**.
Therefore we cannot include categorical variables such as **Species** in the PCA calculation.

In the iris dataset:

- The first **4 columns are numeric measurements**
- The last column (**Species**) is a **categorical variable**

For this reason, we will select only the **numeric columns**.

Select numeric variables

```r
# Select the first four numeric columns from the iris dataset
iris_numeric <- iris[, 1:4]

# Show the first rows of the dataset
head(iris_numeric)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width
## 1          5.1         3.5          1.4         0.2
## 2          4.9         3.0          1.4         0.2
## 3          4.7         3.2          1.3         0.2
## 4          4.6         3.1          1.5         0.2
## 5          5.0         3.6          1.4         0.2
## 6          5.4         3.9          1.7         0.4
```

```r
# Check the structure of the dataset
str(iris_numeric)
```

```
## 'data.frame':    150 obs. of  4 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
```

Why do we do this?

- PCA works only with **numeric data**.
- **Species** is a **categorical variable**, so it is excluded from the PCA calculation.
- Later we will add **Species** back to the dataset to **color the PCA plot**.

4. Run PCA (with scaling)

Now we will run PCA using the `prcomp()` function.

Why do we scale the data?

Our variables (**Sepal.Length, Sepal.Width, Petal.Length, Petal.Width**) are measured in the same unit,
but their **ranges can still be different**.

If we do **not scale the data**:

- Variables with **larger values can dominate the PCA result**.

So we use:

- **scale. = TRUE** → standardizes the variables (makes them comparable).

Run PCA

```r
# Run PCA with scaling
pca_result <- prcomp(iris_numeric, scale. = TRUE)
```

Show PCA variance information

```r
summary(pca_result)
```

```
## Importance of components:
##                           PC1    PC2     PC3     PC4
## Standard deviation     1.7084 0.9560 0.38309 0.14393
## Proportion of Variance 0.7296 0.2285 0.03669 0.00518
## Cumulative Proportion  0.7296 0.9581 0.99482 1.00000
```

Check the PCA summary

The summary tells us how much **variance** each principal component explains.

Usually, **PC1** explains the most variance, then **PC2**, and so on.

How to read the summary (simple)

In the output you will see:

- **Standard deviation** of each PC
- **Proportion of Variance** → how much variance each PC explains
- **Cumulative Proportion** → total variance explained up to that PC

For a **2D PCA plot**, we usually use **PC1 and PC2** because they explain the most variance.

5. Extract PCA Scores

After running PCA, we need the **coordinates of each sample** in the new PCA space.

These coordinates are called **PCA scores**.

They tell us where each sample is located on:

- PC1
- PC2
- PC3
- PC4

These scores are stored inside the PCA object.

Extract PCA scores

```
# Convert PCA scores into a data frame
pca_scores <- as.data.frame(pca_result$x) #Now pca_scores contains the PCA coordinates of each sample.

# Add Species information to the PCA dataset
pca_scores$Species <- iris$Species
```

Add the Species label

Remember that we did not include **Species** in the PCA calculation.

However, we still want to use it to **color the PCA plot**.

Now our dataset contains:

- **PCA coordinates** (PC1, PC2, PC3, PC4)
- **Species label**

This allows us to **visualize the PCA results by species**.

6. Plot PCA (PC1 vs PC2)

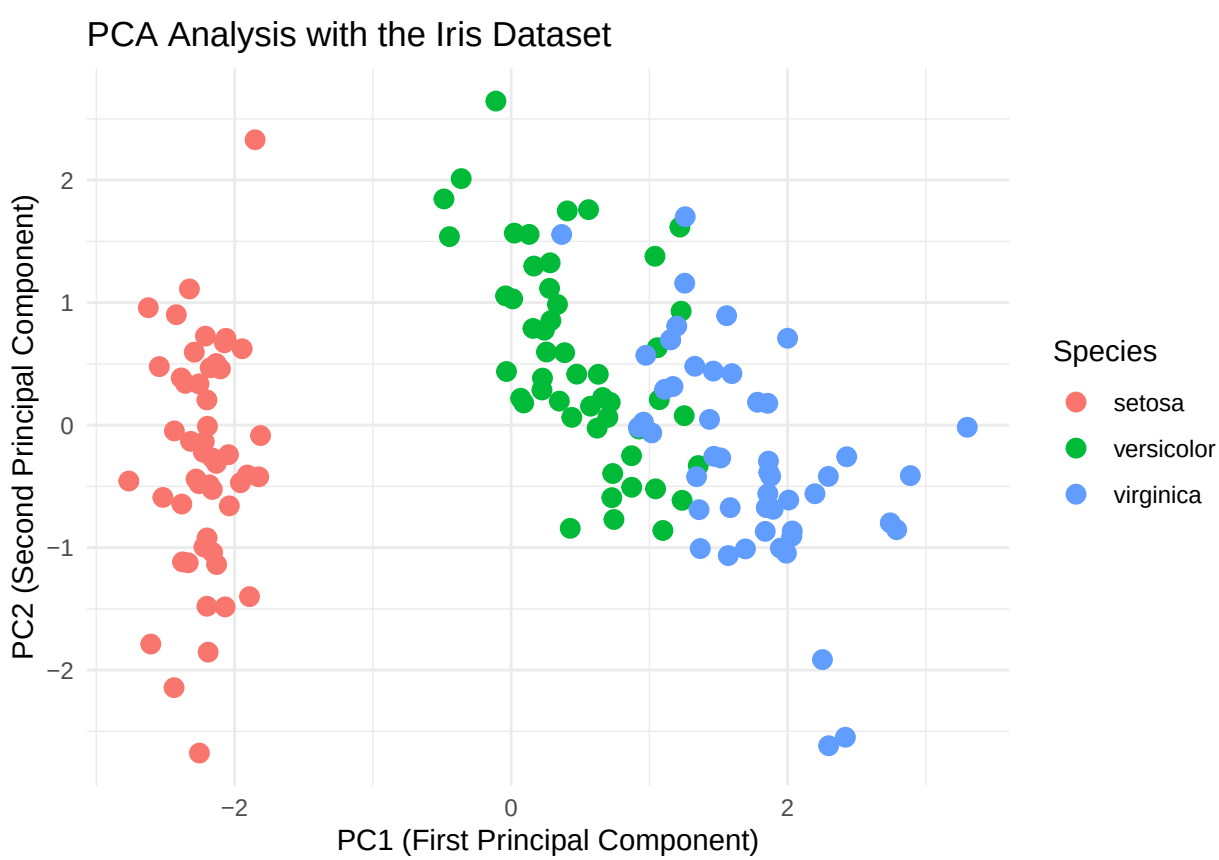
Now we will visualize the PCA results using a **2D scatter plot**.

We will plot:

- **PC1** on the x-axis
- **PC2** on the y-axis
- color points by **Species**

Create the PCA plot

```
# PCA plot: PC1 vs PC2
ggplot(pca_scores, aes(x = PC1, y = PC2, color = Species)) +
  geom_point(size = 3) +
  xlab("PC1 (First Principal Component)") +
  ylab("PC2 (Second Principal Component)") +
  ggtitle("PCA Analysis with the Iris Dataset") +
  theme_minimal()
```



How to interpret the PCA plot (simple)

The PCA plot helps us understand **how similar or different the samples are**.

- Points that are **close together** represent samples that are **similar**.
- Points that are **far apart** represent samples that are **different**.
- If species form **separate clusters**, PCA can help **distinguish them**.

In the iris dataset:

- **setosa** samples are usually **clearly separated** from the other species.
- **versicolor** and **virginica** are **closer together** and may **partially overlap**.

How to read a PCA plot

- The **x-axis (PC1)** represents the direction with the **largest variance** in the dataset.
- The **y-axis (PC2)** represents the **second largest variance**.
- Each point represents **one sample** in the dataset.
- Colors indicate **different groups or species**.